第 7 章練習　死結

> 本檔只含題目。請先自行建立狀態或變數表，再作答；解答在獨立檔案。

OS07-E01　四必要條件

兩個 process 各持有一個不可共享 lock，並互相等待另一把 lock。

1. 列出四個條件在此例中的具體對應。
2. 若把其中一把 lock 改成可搶占，deadlock 是否必定消失？
3. 若有 cycle，但另有可釋放的 instance，能否直接斷言 deadlock？

OS07-E02　安全狀態

Available=(2,1,0)。Allocation：P1=(1,0,0)、P2=(0,1,0)、P3=(0,0,1)。Need：P1=(1,1,0)、P2=(2,1,0)、P3=(3,0,0)。

1. 找出一個安全完成順序。
2. 執行 P1 後若釋放它已持有的 (1,0,0)，Available 變成多少？
3. 若 P1 的 Need 改成 (2,1,1)，此時是否還能完成全部？先檢查所有 Max=Allocation+Need 不超過系統總量。

OS07-E03　Victim 選擇

資源被 preempt 後，process 已修改共享樹的一半節點。

1. 只收回 lock 為何不安全？
2. 何種復原單位使 snapshot 回到一致？
3. 如何在避免 starvation 下選 victim？

OS07-E04　Resource graph 的 cycle 判準

R1 有兩個 instances；P1 持有其中一個並等 R2。P2 持有唯一 R2、等 R1。P3 持有另一個 R1 且沒有其他需求。

1. 寫出 request 與 assignment edges。
2. 圖中是否存在 cycle？所有四必要條件是否成立？
3. 此狀態是否 deadlocked？用可完成順序說明。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 7 章練習　死結 （續）

OS07-E05　Banker request 完整試算

> 使用課本 T0：Total=(10,5,7)、Available=(3,3,2)；Allocation P0..P4=(0,1,0)、(2,0,0)、(3,0,2)、(2,1,1)、(0,0,2)；Need=(7,4,3)、(1,2,2)、(6,0,0)、(0,1,1)、(4,3,1)。P1 請求 (1,0,2)。
>
> 1. 先執行 request 演算法的 claim 與 availability 檢查。
> 2. 建立 trial state 的 Available、Allocation1、Need1。
> 3. 逐列執行 safety algorithm 並給出安全序列。
> 4. 若 allocation 導致 unsafe，request 流程最後要做什麼？

OS07-E06　Multi-instance detection 與 recovery

Total=(7,2,6)、Available=(0,0,0)。Allocation：P0=(0,1,0)、P1=(2,0,0)、P2=(3,0,3)、P3=(2,1,1)、P4=(0,0,2)；Request：P0=(0,0,0)、P1=(2,0,2)、P2=(0,0,1)、P3=(1,0,0)、P4=(0,0,2)。

1. 依 detection algorithm 逐步列 Work、可完成的 process 與 Finish。
2. 哪些 process 在最終仍 deadlocked？
3. 若終止 P1，資源如何回到 Available，後續能否解除其餘等待？
4. 為何 recovery 不能只奪回 lock 而略過 rollback？

OS07-E07　同一個環，兩種不同結局

> R1 有兩份、R2 有一份；向量順序 (R1,R2)。初始 Available=(0,0)。P1、P2、P3 的 Allocation 分別為 (1,0)、(0,1)、(1,0)，目前 Request 分別為 (0,1)、(1,0)、(0,0)。滿足目前請求後，每個 process 均可完成並歸還全部持有資源；沒有提前釋放、搶占或取消。
>
> 1. 畫出資源實例、持有箭頭與請求箭頭，指出一個環。能否只憑此環判定死結？
> 2. 從 Work=(0,0) 逐步做 detection reduction，列每步 Request≤Work 的檢查、加回量及新 Work。模擬結束時，真實 Available 必然是 (2,1) 嗎？
> 3. 依此完成順序實際執行，逐次列「授予」與「完成歸還」後的 Available；不得省略授予造成的扣除。
> 4. 只把 P3 的 Request 改為 (0,1)。列出無法開始的三個向量比較，判定死結集合，並說明原例中缺少的哪條邊現在出現了。

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study